# Robot Foundation Model

Jie Tan, Peng Xu
Google DeepMind

Oct 26, 2026

# Goal

What is Robot Foundation Model?

Why Robot Foundation Model?

How is it built and applied in Robotics?

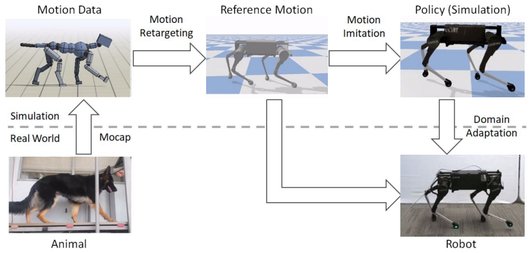
# In the last decade…

## Reinforcement Learning

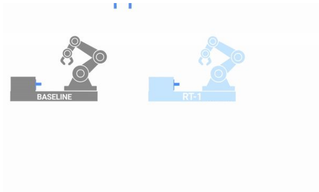
## Robot Foundation Model



Arm Farm, 2016



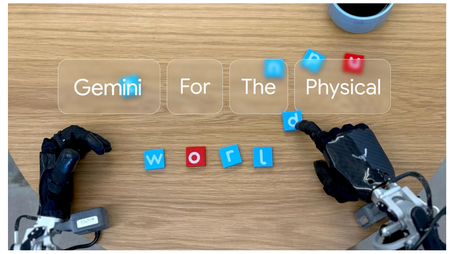

Imitating Animals, 2020



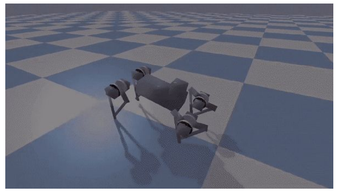

RT-1, 2022



RT-2, 2023




Gemini Robotics, 2025





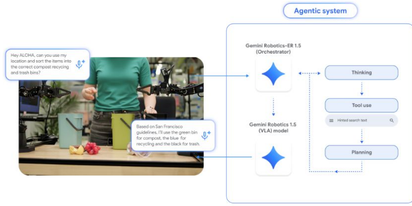
Sim-to-Real, 2018




SayCan, 2022



RT-x, 2023




Gemini Robotics 1.5, 2025

# RFM for Pupper

## What You Built in CS 123 so far

**The Agile Body & Fast Reflexes**

- **Hardware & Kinematics:** Motors, leg geometry, and foot placement
- **Simulation:** Visualize and accelerate RL training
- **RL Walking Gait:** Learning to walk autonomously, from scratch, and transfer policy from sim to real

*Limitation: Pupper walks great—but has no idea about our world, and cannot interact with people in a fun way!*

+

## What an RFM Unlocks

**Vision, Language & Planning**

- **Open-vocabulary vision:** Finds everyday objects without brittle color filters
- **Natural language instruction:** Understands "fetch the toy near the backpack"
- **Common-sense reasoning:** Breaks vague goals into sensible steps

*Result: Pupper can talk to you, understand you, and execute more complex tasks!*

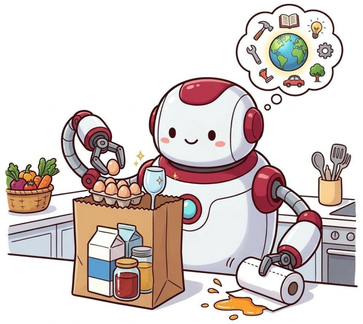
# RFM on Pupper



# Why Robot Foundation Model?

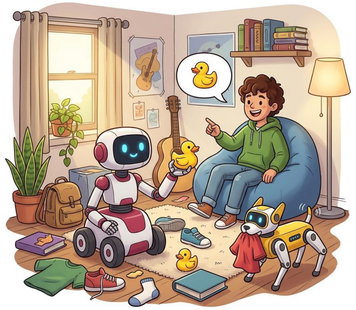
## 01 Built-In World Common Sense

Inherits human knowledge from billions of web pages and images: it already knows wine glasses shatter, spilled drinks need paper towels, and heavy items go at the bottom of a grocery bag.



## 02 Generalization to the Messy Real World

Handles unseen objects, cluttered backgrounds, changing lighting, casual human instructions, and even new robot bodies with minimal extra training.



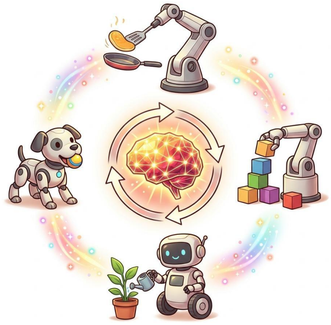
## 03 Little Hand-Crafted Engineering

The bitter lesson: “General purpose methods that scale with computation consistently outperform human designed, domain specific knowledge in AI”

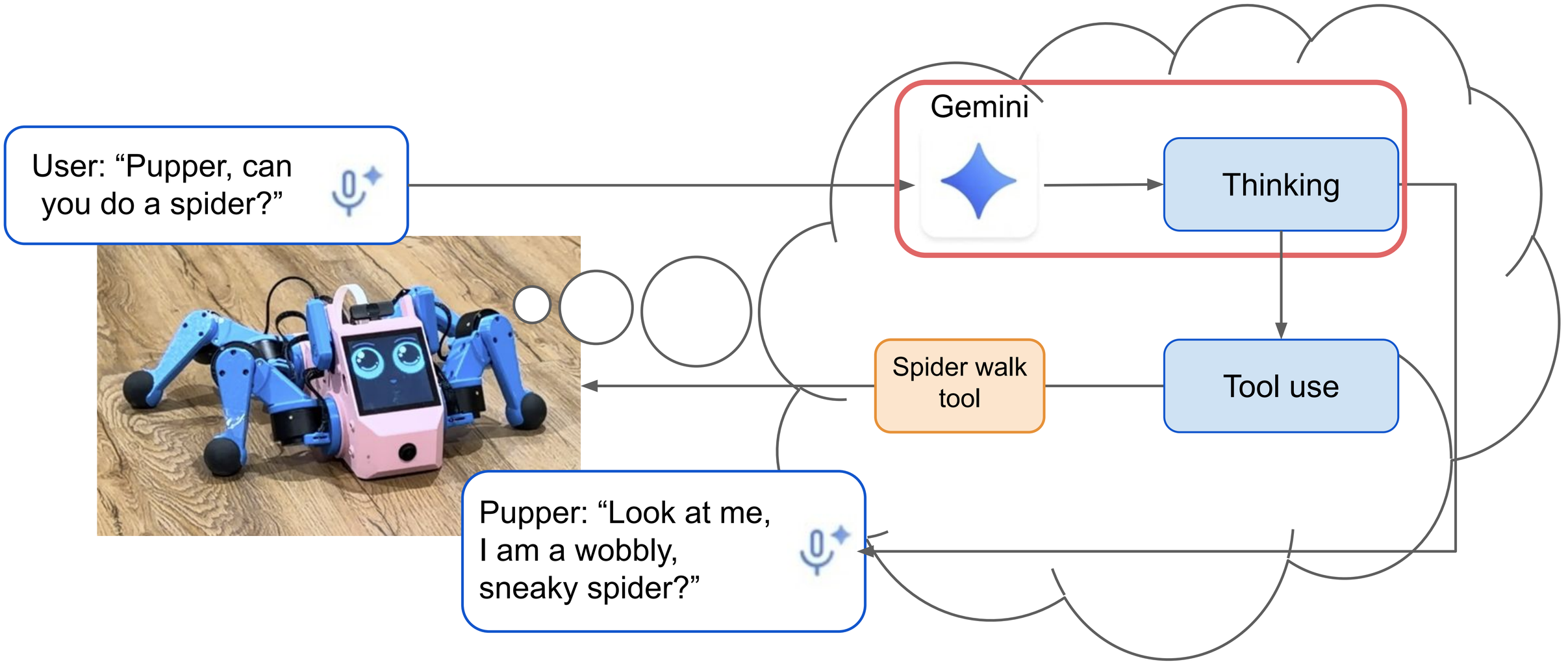


## 04 A Shared Data Flywheel

Instead of every project starting from zero, new robot demonstrations improve one shared foundation that benefits future tasks.



# How does it work?



# Vision Language Model Basics





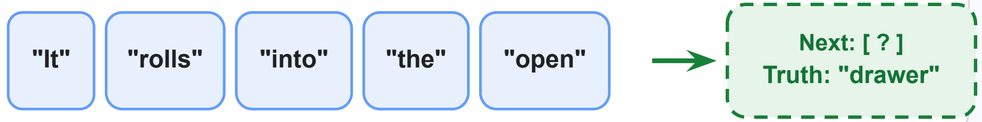

# Vision-Language Models: Two Big Ideas

**Big Idea 1: Self-Supervised Training Objective**

## Next-Token Prediction

*Turn the entire Internet into one prediction game*

**Every Web Sentence → Predict the Next Word**




**Result:** Predicting the next word at web scale compresses **common sense, physics & reasoning** into model weights!

+

**Big Idea 2: Scalable Neural Architecture**

## Transformer Architecture

*One unified sequence engine for text & vision*

- Text Tokenization
- Vision Transformer (ViT)
- Attention Mechanism
- Scaling Laws

**Two Big Ideas: Next-Token Prediction** supplies trillions of free labels; the **Transformer** scales them into intelligence!

# Big Idea 1: The Internet as Next-Token Prediction

**1. Raw Internet Text**

**Wikipedia & Science Books**

*“Glass is brittle and shatters when dropped onto a hard surface...”*

**Everyday Stories & Guides**

*“A ripe orange rolls into the open drawer.”*

**Tutorials, Dialogues & Code**

*“Step 1: open top drawer.*
*Step 2: place orange inside...”*

# Big Idea 1: The Internet as Next-Token Prediction

**1. Raw Internet Text**

**Wikipedia & Science Books**
*“Glass is brittle and shatters when dropped onto a hard surface...”*

**Everyday Stories & Guides**
*“A ripe orange rolls into the open drawer.”*

**Tutorials, Dialogues & Code**
*“Step 1: open top drawer.*
*Step 2: place orange inside...”*

**Sliding Window**

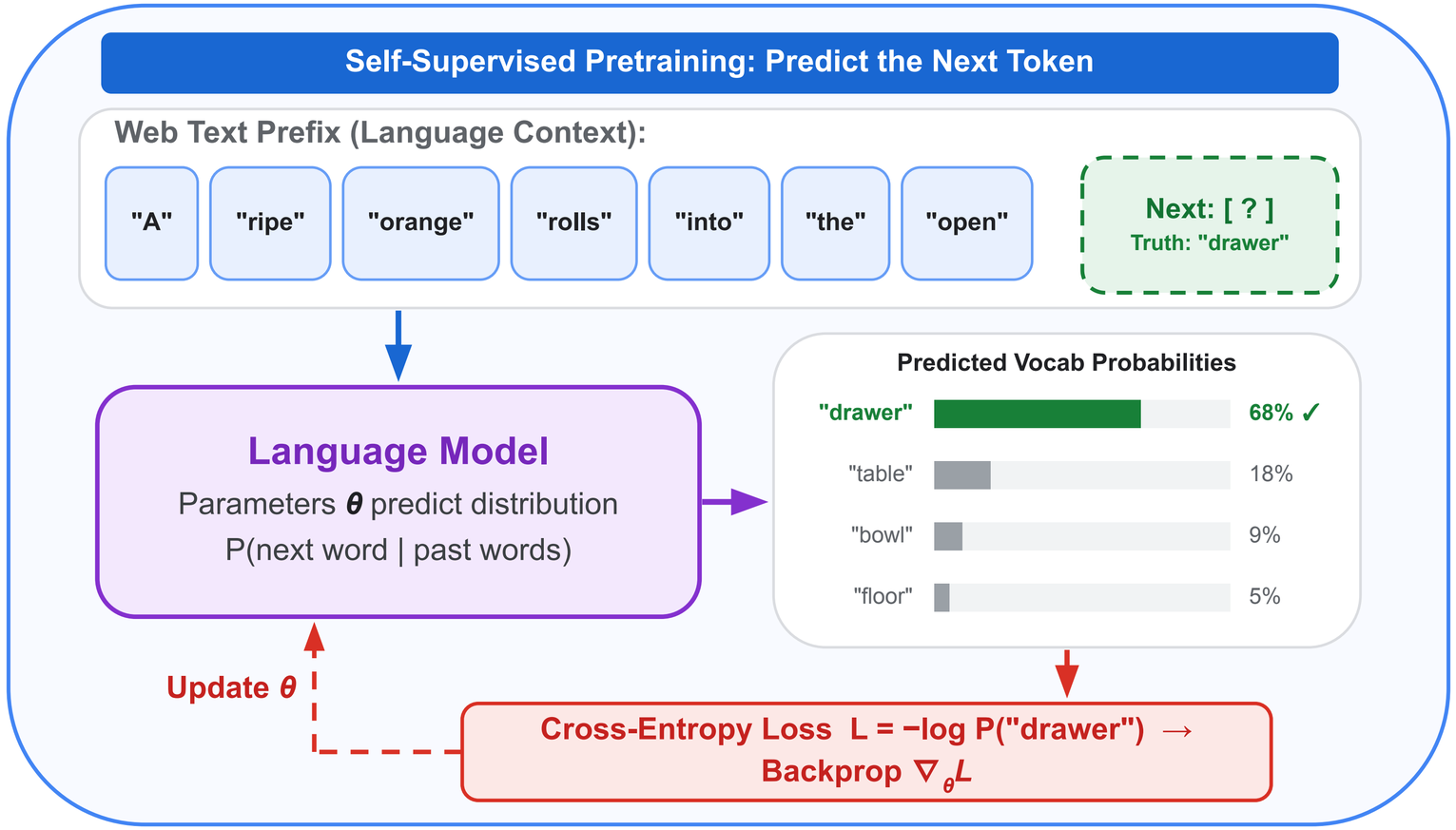
1 sentence = 7 training pairs!

**2. Self-Supervised (Input Context → Next Word) Training Examples**

| Input Context (Past Words) | Free Label |
|---|---|
| "A" | "ripe" |
| "A" "ripe" | "orange" |
| "A" "ripe" "orange" | "rolls" |
| "A" "ripe" "orange" "rolls" "into" | "the" |
| "A" "ripe" "orange" "rolls" "into" "the" "open" | "drawer" |

**Self-Supervised Trick:** Slide a window across **any text on the Internet** — every word supervises the words before it!

# Pretraining Engine: Next-Token Prediction at Web Scale

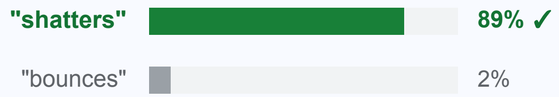



**Self-Supervised Magic:** Every **next word** on the Web is a **free label** — scaling to **trillions of tokens!**

# Why Next-Word Prediction Learns World Knowledge

### 1. Physical Understanding

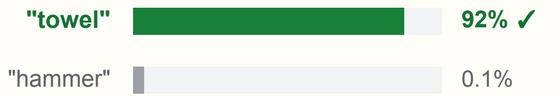
*“A heavy bowling ball drops onto a fragile wine glass, so it ___”*




*Learns fragility, weight & gravity*

### 2. Everyday Common Sense

*“Orange juice spilled all over the kitchen counter — grab a paper ___”*




*Learns everyday object affordances*

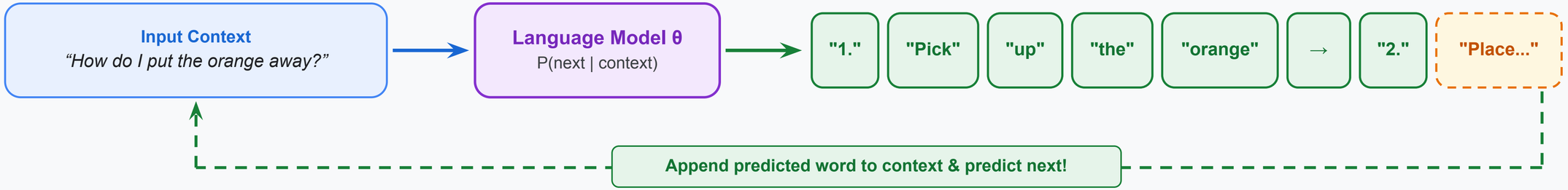
### 3. Multi-Step Task Logic

*“Goal: tidy counter. Step 1: open top drawer. Step 2: pick up ___”*




*Learns sequential step-by-step plans*

**At Test Time — Autoregressive Loop:** Predict 1 next word, append it to the input, and repeat!




**Compression = Understanding:** Predicting trillions of words distills **physics, common sense & planning** into θ!

# Big Ideas: Transformer Architecture

**Big Idea 1: Self-Supervised Training Objective**

## Next-Token Prediction

*Turn the entire Internet into one prediction game*

**Every Web Sentence → Predict the Next Word**




**Result:** Predicting the next word at web scale compresses **common sense, physics & reasoning** into model weights!

+

**Big Idea 2: Scalable Neural Architecture**

## Transformer Architecture

***One unified sequence engine for text & vision***

- **Text Tokenization**
- **Vision Transformer (ViT)**
- **Attention Mechanism**
- **Scaling Laws**

**Two Big Ideas: Next-Token Prediction** supplies trillions of free labels; the **Transformer** scales them into intelligence!

# Vision Language Model

***Visual input***
*One or multiple images (different views or history)*



***Prompt***
*"What action should the robot take to place orange into top drawer?"*

VLM

***Response:*** *"I need to move my gripper by (30, 24, -9) centimeters, and close my gripper"*

***Think:*** *"Write a program to detect where the orange is:*

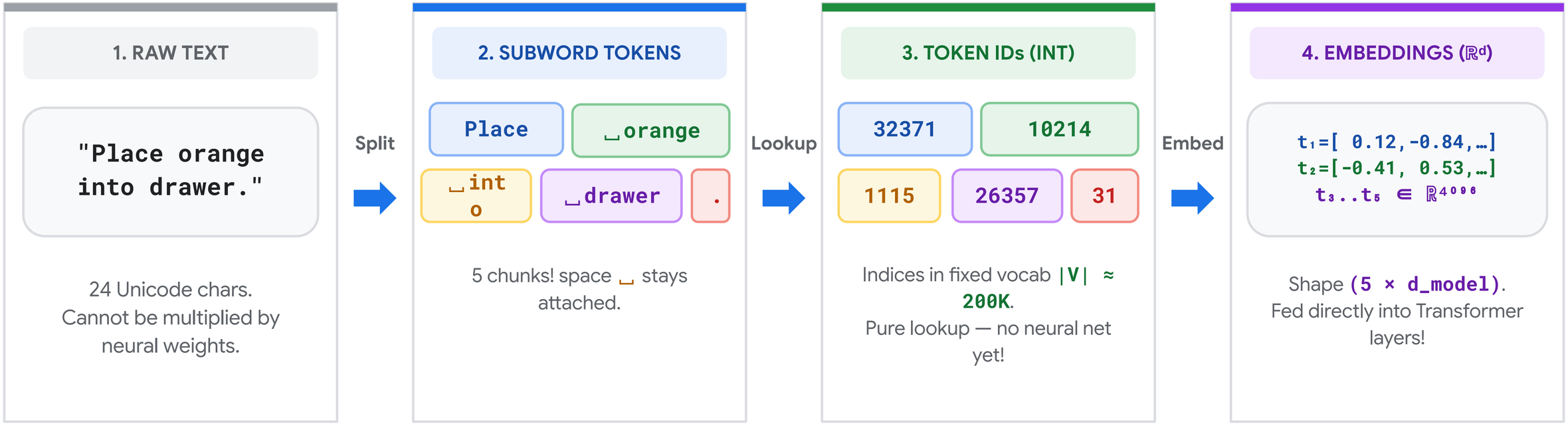
```
def orange_detection(img):
    ...
    return bounding_box
```

*Now let me run this program and get the position of the orange..."*

# Why Tokenization?

**The Core Problem:** Transformers only perform matrix multiplications, not on text.

**Tokenization** deterministically splits raw text into discrete **tokens** from a fixed vocabulary **V**, mapping strings ↔ embedding vectors.




**Why "␣into" vs. "into"?**
**Why not feed integer Token IDs directly?**

# From Pixels to Tokens: Vision Transformer (ViT)

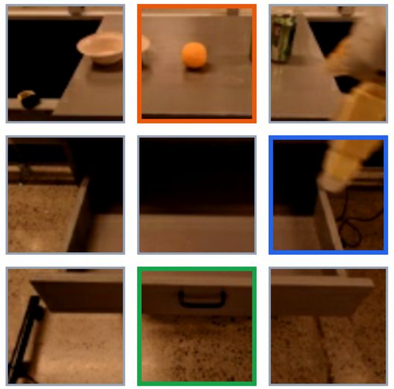
**Why not 1 pixel = 1 token?** A 224×224 image has **50,176 pixels** — far too long for attention! Just as tokenization groups letters into words, **ViT groups pixels into patches ("visual words").**

### ① Camera Image



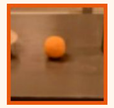
**2D RGB Pixel Grid**
224 × 224 × 3 floats
(raw camera frame)

### ② Split into Patches

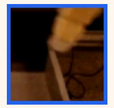


**Non-overlapping tiles**
e.g. 16×16 px per patch
→ 196 visual patches

### ③ 2D Pos + ViT Encoder



**Pos(1,2) [orange]**
$v_1$=[0.82, -0.31, …, 0.44]



**Pos(2,3) [gripper]**
$v_2$=[-0.14, 0.67, …, 0.12]



**Pos(3,2) [drawer]**
$v_3$=[0.45, 0.19, …, -0.58]

**2D Position Embedding** preserves location of patch
**Visual token** preserves semantics of the patch

**Key Intuition:** To a Transformer, an image is no longer a 2D pixel grid — it is **a 1D sequence of float vectors**, just like text embeddings!

# Turning an LLM into a VLM: Token Interleaving



Vision Encoder

Visual Vecs
$[v_1 \dots v_9]$
D floats each

"Place orange into drawer"
Token IDs [32371, …]

Text Token Embedding
(lookup table)

Word Vecs
$[t_1 \dots t_4]$
D floats each

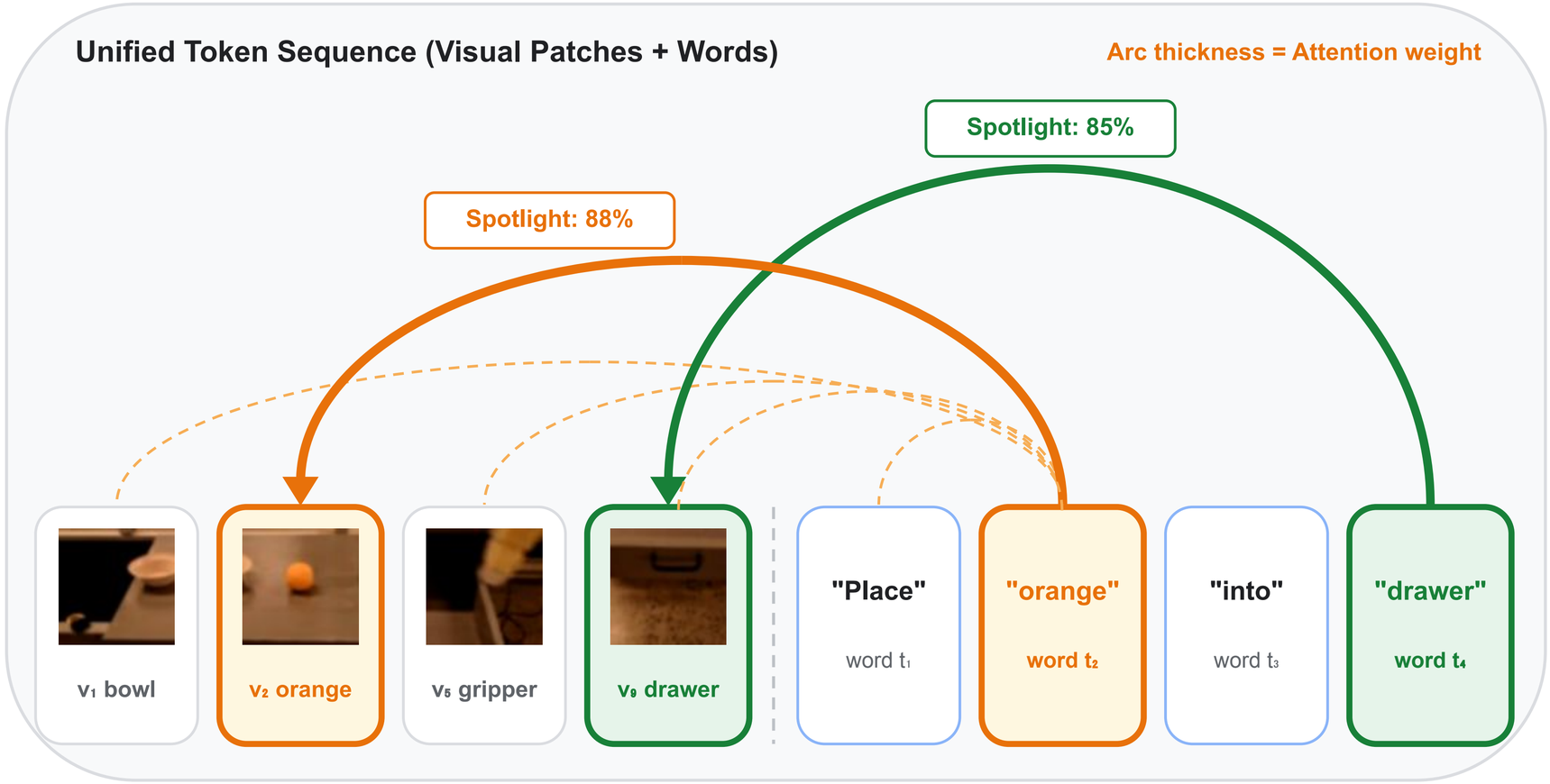
| <img start> |  $v_1$ bowl |  $v_2$ orange | … |  $v_9$ drawer | <img end> | $t_1$ "Place" | $t_2$ "orange" | $t_3$ "into" | $t_4$ "drawer" |
|---|---|---|---|---|---|---|---|---|---|

LLM Transformer Backbone

Next-Token Prediction

**LLM → VLM:** Interleaving visual & text embeddings turns an LLM into a **VLM** with zero change to LLM network architecture.

# Attention: A Dynamic Spotlight Across Tokens




## What Changes?

### 1. Before Self-Attention

"orange" = generic word

No idea where the orange sits in the camera view!

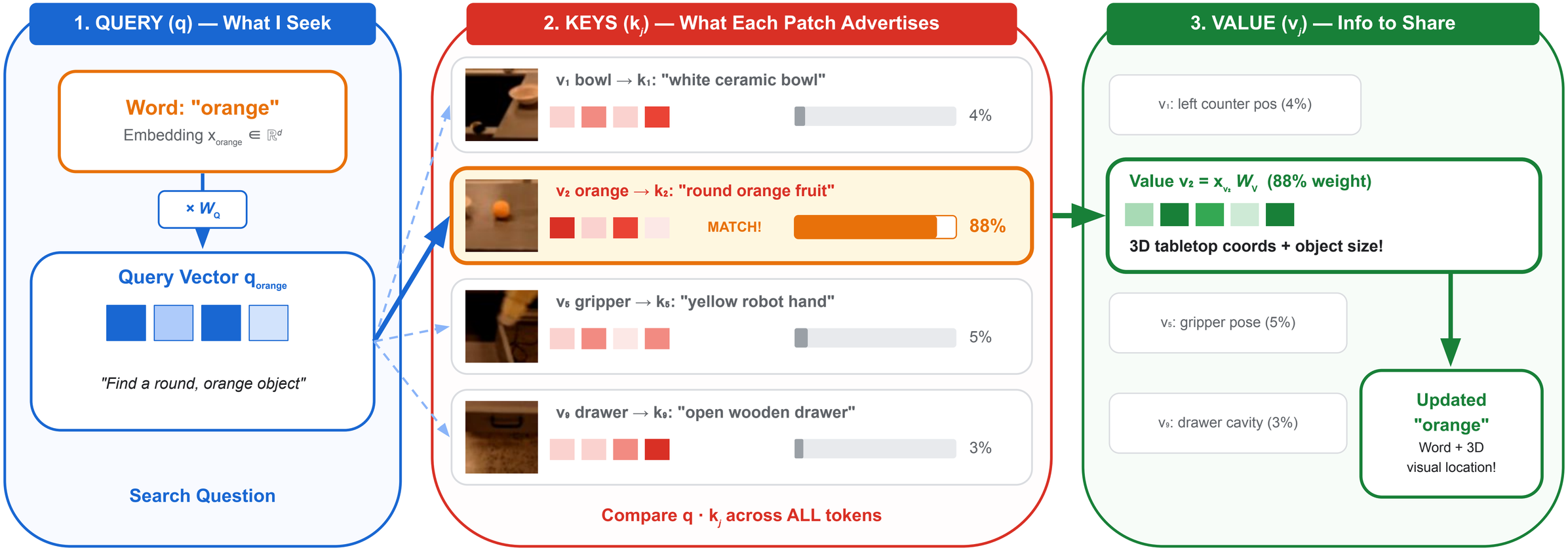
Self-Attention

### 2. After Self-Attention

"orange" + patch $v_2$ info

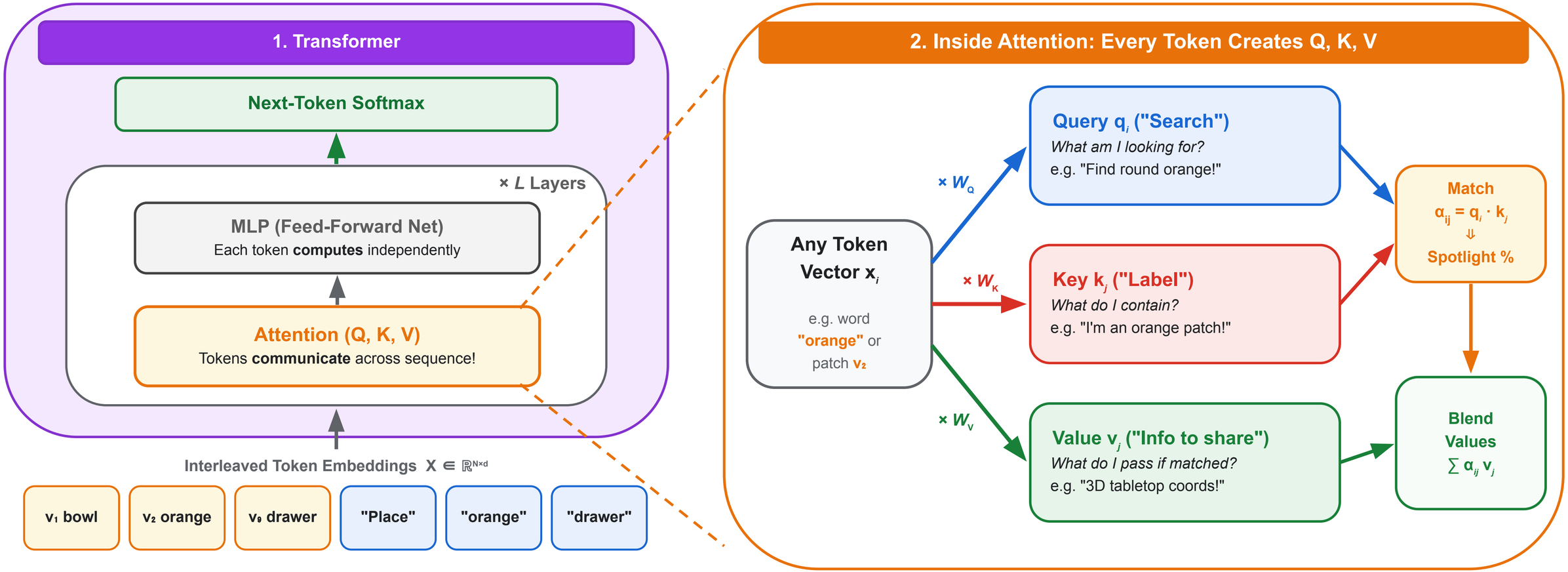
Grounded to 3D tabletop position & visual appearance!

# Inside the Spotlight: Query, Key, and Value

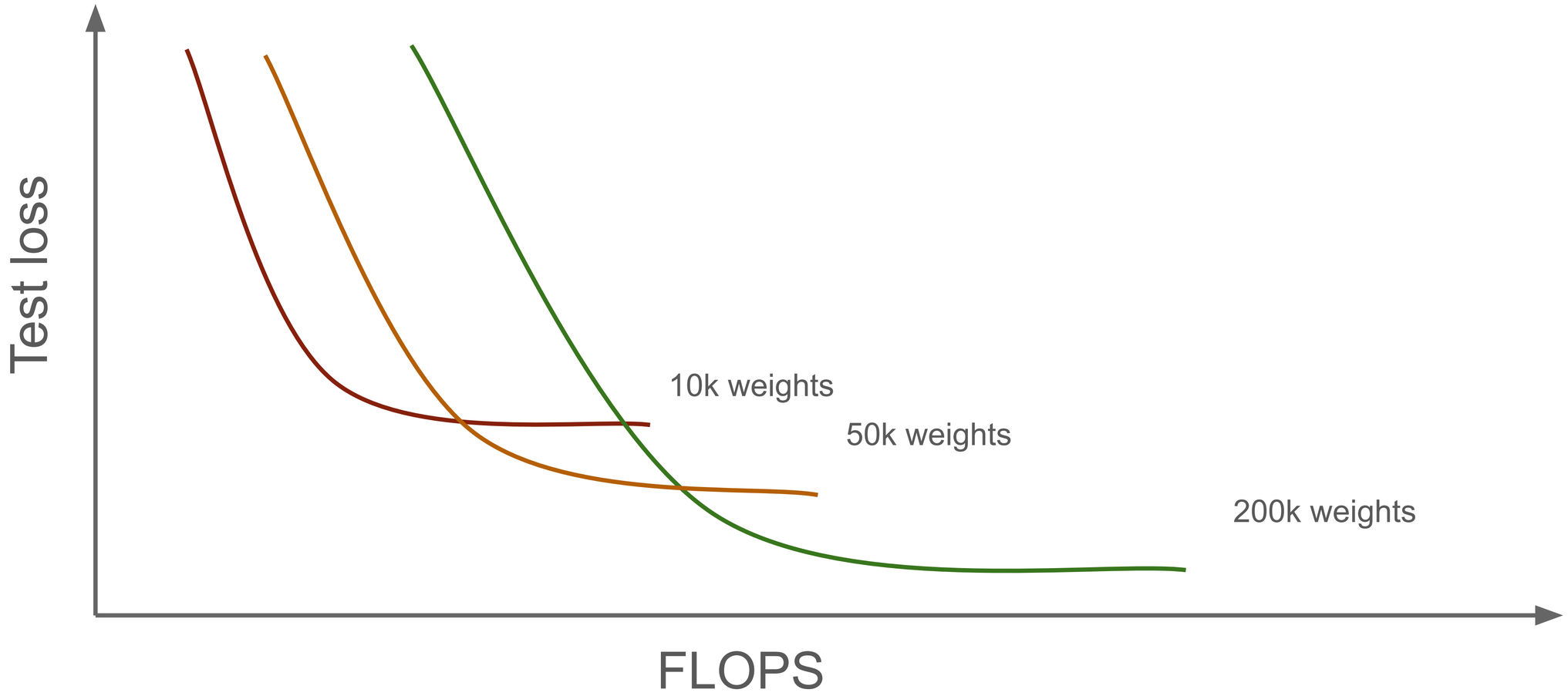



**Learned Lookup:** Matrices $W_Q$, $W_K$, $W_V$ train every token to **ask**, **advertise**, and **share** features.

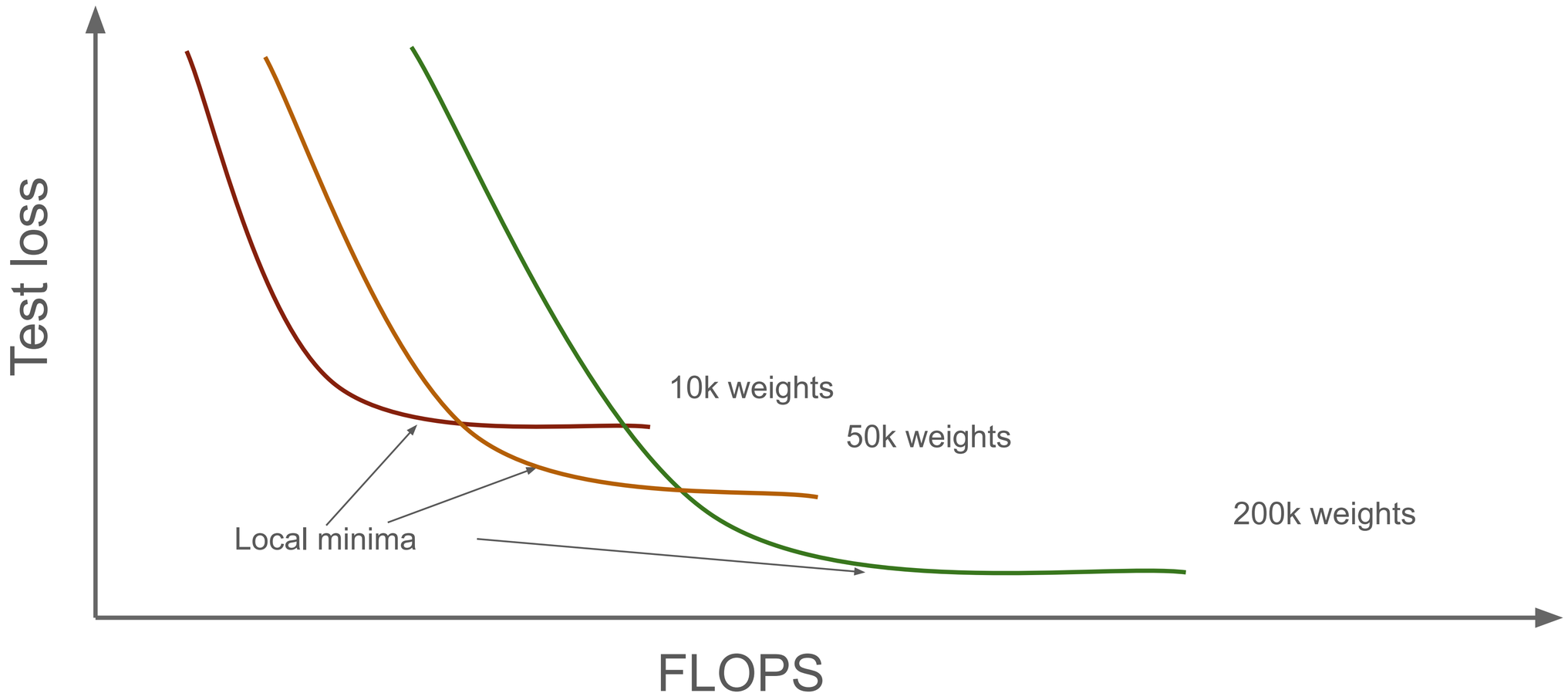
# Inside the Transformer: Stacked Blocks & Q-K-V



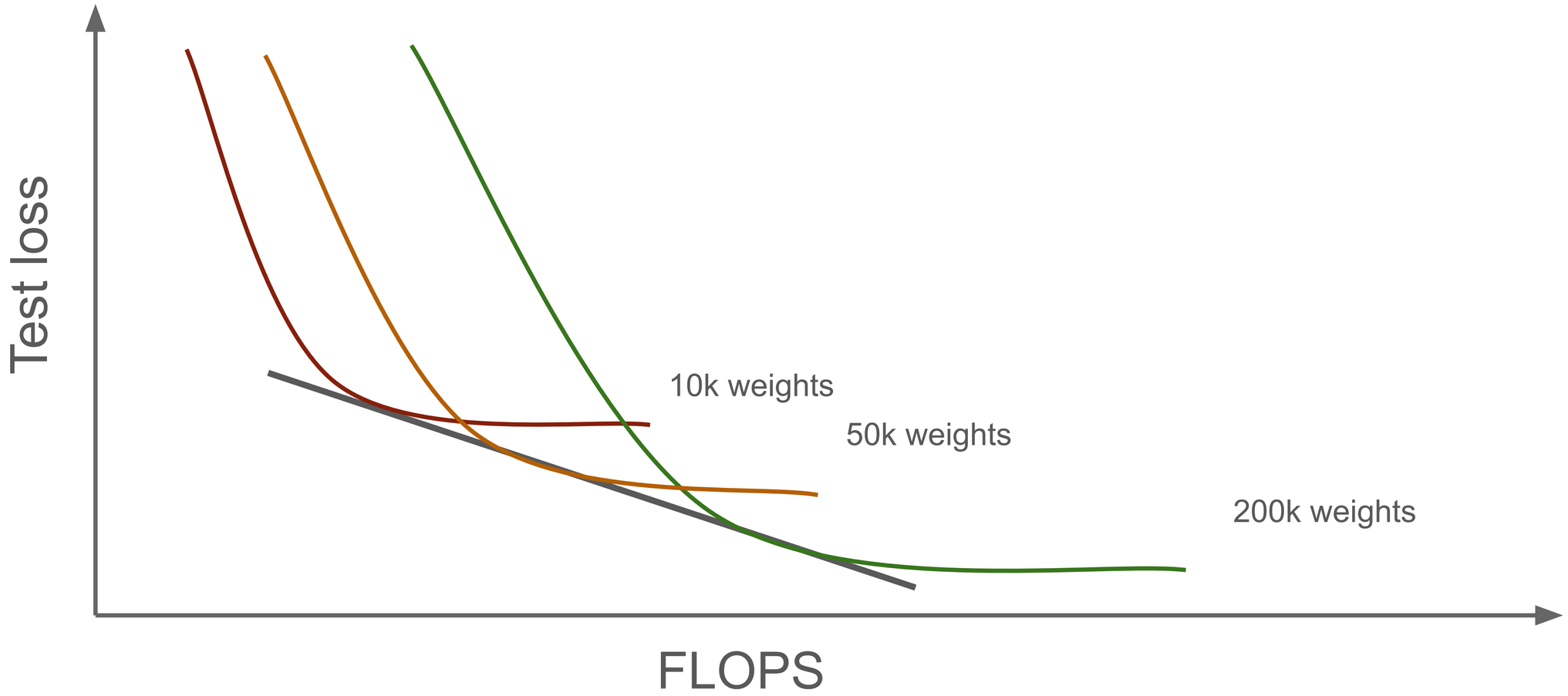
# Why Transformer Architecture: Scaling Laws



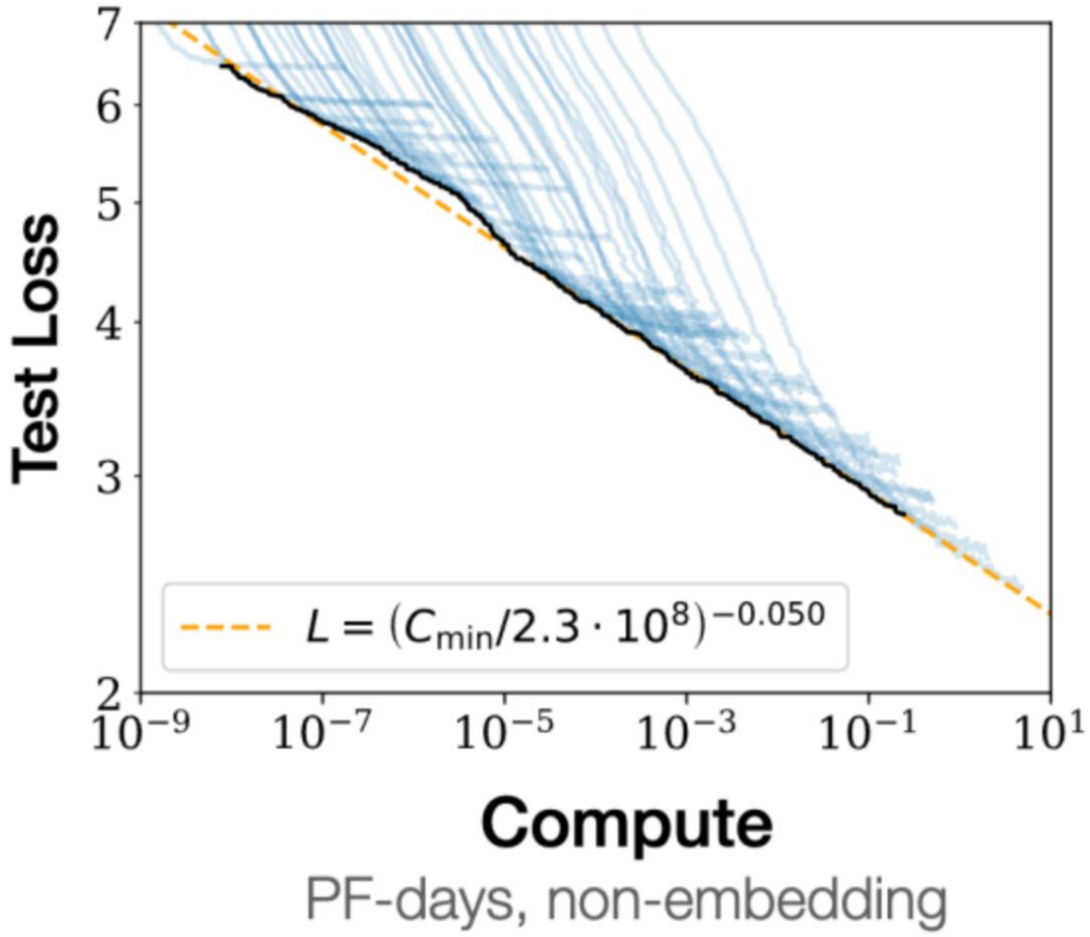
# Why Transformer Architecture: Scaling Laws



# Why Transformer Architecture: Scaling Laws

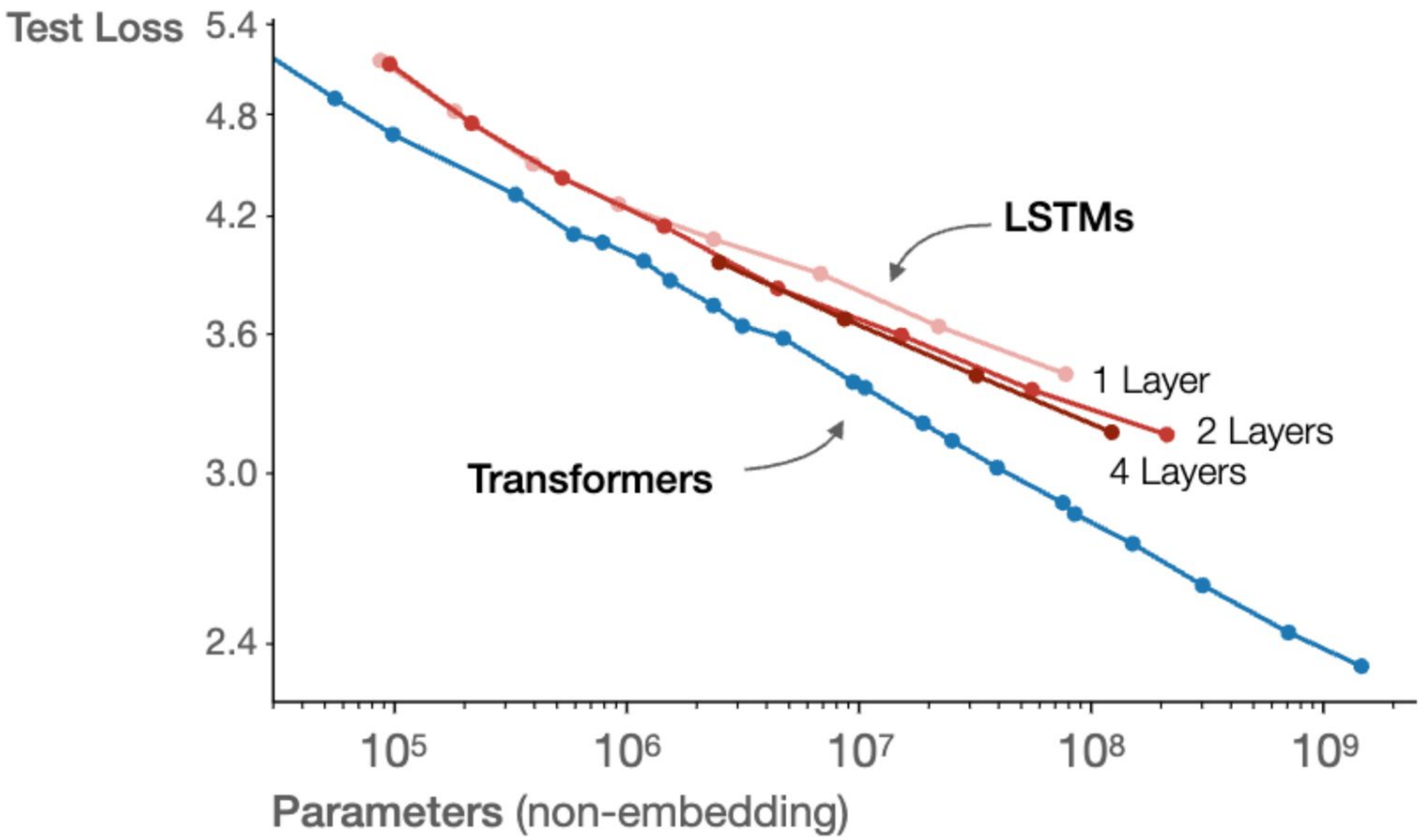


# Why Transformer Architecture: Scaling Laws



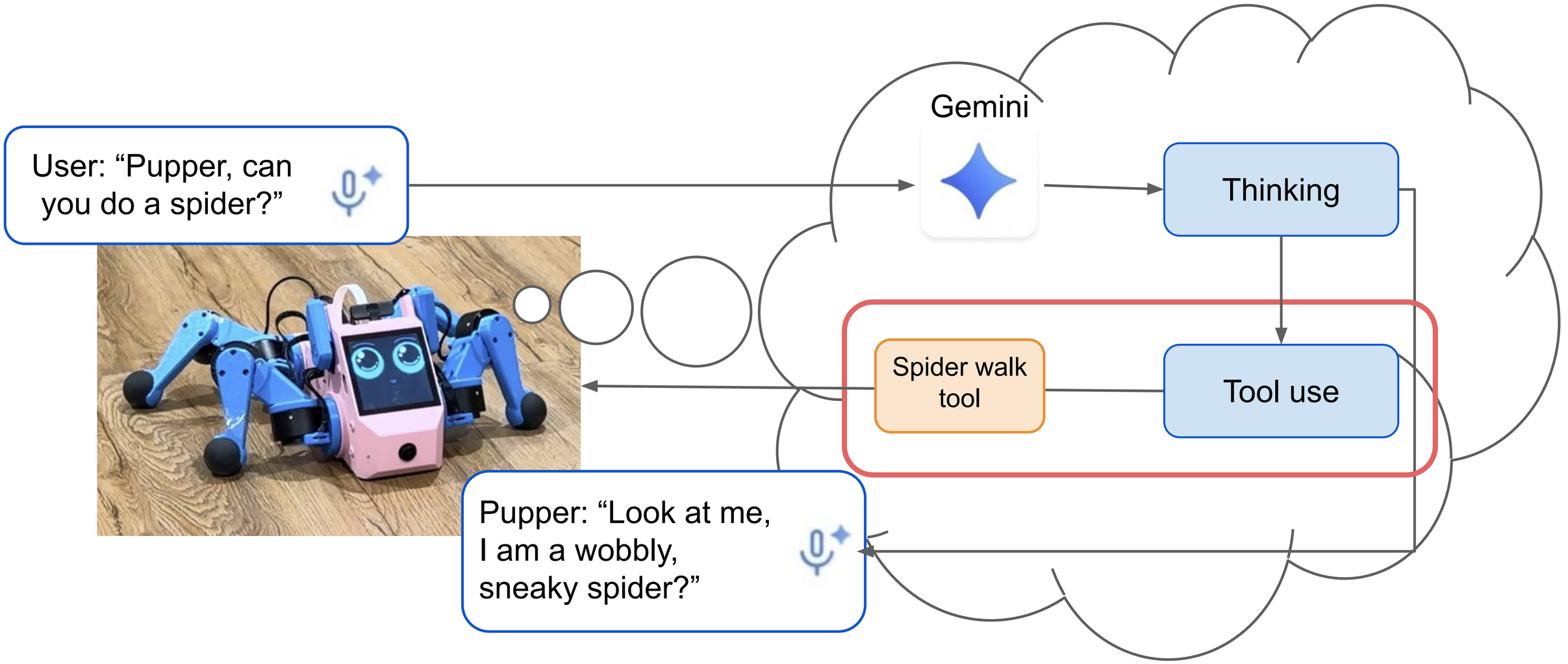

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

# Why Transformer Architecture: Scaling Laws




**Scaling:** Transformer models have better scaling property than other architectures: **more data, bigger model, better performance**!

# Agent and Harness



# Agent and Harness

Are they just buzzwords?

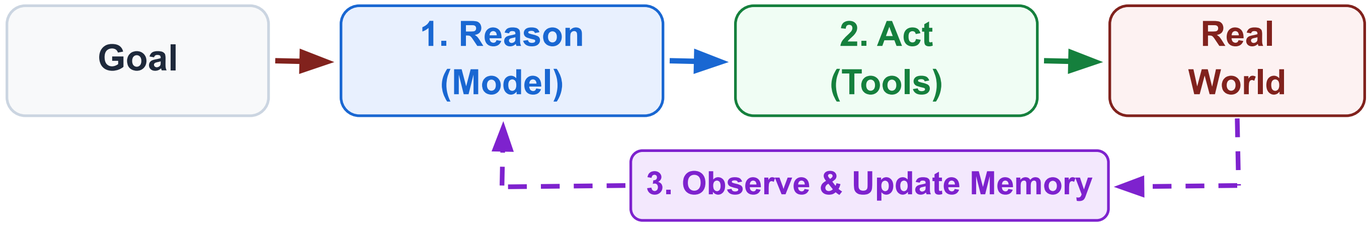
# Model vs. Agent

## Model




- One pass, then stops
- Tokens only (✕ no real world interaction)
- Stateless (✕ no memory)

## Agent




**1. Iteration loop** — Reason → Act → Observe

**2. Tools** — Call cloud APIs, Run code, Read sensors etc

**3. Memory** — Working memory + persistent storage

**Key Distinction:** **Model = 1 pass next token prediction** • **Agent acts, observes & adapts in a loop**

Reference: ReAct (Yao et al., ICLR 2023)

# Agent in Daily Life → Robotics

## 1. Coding Agent

Claude Code • Cursor • Gemini CLI



**Write a mobile app**

- **Tools:** `edit_file(), pytest`
- **Loop:** Generate code →edit code file → run test → read error msg

## 2. Personal Agent

Gemini App • Phone Assistant



**Book a room for my meeting with Jie**

- **Tools:** `calendar(), team_list()`
- **Loop:** Find who Jie is -> find meeting with Jie → find a open room → book it.

## 3. Robot Dog Agent

Pupper



**Approach ball and sit**

- **Tools:** `walk(), animate()`
- **Loop:** Think what is the best action -> walk -> animate -> robot state.

**Same Architecture:** `edit_file() & pytest` ⟶ `animate() & walk()`

# Agent and Harness

## Agent

**Acts in the Real World**

- Iterates toward a goal
- Decides on actions
- Adapts from feedback

***Coding • Personal • Robotics***

=

## Model

**The Reasoning Brain**

- Next-token predictor
- Web-scale knowledge
- Reasoning from multimodal input

***Neural Networks***

+

## Harness

**Everything Else**

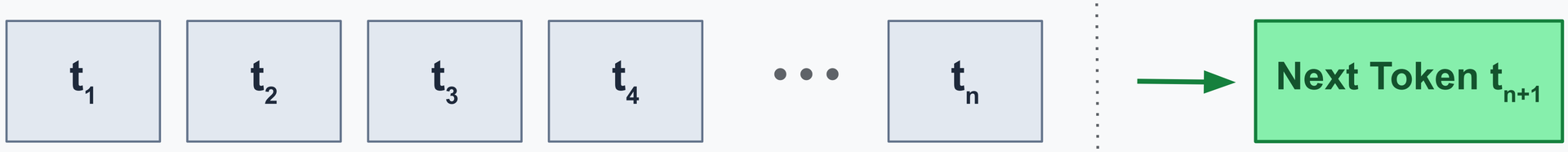
- Iterative control loop
- Memory
- Tools

***Software***

**Nothing Fancy:** **Agent = Model + Harness** - Model makes decisions, harness makes it happen and provides feedbacks

# Starting from Next-Token Prediction

**One Simple Principle:** **Predict the next token**




**Model**

- Write document / code
- Summarize reports
- Detect objects

→

**Build 1: Simple Agent**

➕ Live conversation in text.
➕ Memorize conversation history.

→

**Build 2: Robot Dog Agent**

➕ Robot function call.
➕ Audio input and output.

**Roadmap:** Let's trace **Build 1 (Simple Agent)**, then upgrade it to **Build 2 (Robot Dog Agent)**!

# Example 1: Inside a Simple Agent

Legend: ■ User Input ■ Underlined = Added by Harness ■ Generated by Model ■ Bold == new at this step.

### 1. System Prompt

System: You are Pupper, a friendly robot dog.

### 2. User Input

System: You are Pupper, a friendly robot dog.
**User: Hello, what is your name?**
**Pupper:**

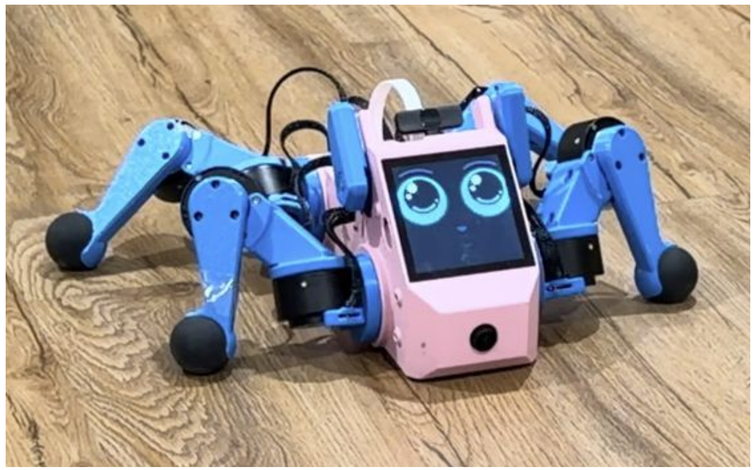
### 3. Next-Token Prediction until reaches stop token

System: You are Pupper, a friendly robot dog.
User: Hello, what is your name?
Pupper: **I am Pupper, a friendly robot dog.<STOP>**

### 4. Move to next turn and Append history

System: You are Pupper, a friendly robot dog.
User: Hello, what is your name?
Pupper: I am pupper, a friendly robot dog.
**User: Can you help control my robot?**
**Pupper:**...

# Simple Agent → Robot Dog Agent



**Brain + Body**

Harness —calls→ Tools

## Goal: Put Our Agent on Pupper

🗣️ "Bark!" → 🔊 "Woof! Woof!" (Live Audio)

🗣️ "Do a spider!" → 🦿 spider_walk() (Calls Function)

### Upgrade 1: Live Audio I/O

- Connect speaker and mic.
- Built into Gemini Live API

Gemini LIVE

### Upgrade 2: Call Functions

- Token alone don't spin motors
- **Harness declares functions.**
- **Harness runs functions!**

# Example 2: Function Calling Under the Hood

## Complete Context Transcript

```
# 1. Harness declares tool + units in System prompt
System: You are Pupper, a friendly robot dog.
Available function: spider_walk(vel, time): vel in m/s
(float), time in sec (float).

# 2. Action turn → Model predicts <function_call>
User:   "Do a spider!"
Pupper: function_call[spider_walk(vel=1.0, time=1.0)]

# 3. Harness runs Python on robot & injects Result
Harness: spider_walk(vel=1.0, time=1.0)
Result: {"status": "ok", "moved_m": 1.0}
Pupper: "I have moved forward 1 meter!"
```

## How the Protocol Works

**1. Declare Function Spec**
Harness puts function args & units in System

**2. Model Emits <function_call>**
Still just next-token text prediction!

**3. Harness Runs Python**
Executes run_forward(1.0, 1.0) on hardware

**4. Return Result → Reply**
Appends JSON status; model confirms aloud

# Example 2: Function Calling Under the Hood

## Complete Context Transcript

```
# 1. Harness declares tool + units in System prompt
System: You are Pupper, a friendly robot dog.
Available function: spider_walk(vel, time): vel in m/s
(float), time in sec (float).

# 2. Action turn → Model predicts <function_call>
User:   "Do a spider at 10 meter per second!"
Pupper: function_call[spider_walk(vel=10.0, time=1.0)]

# 3. Harness runs Python on robot & injects Result
Harness: spider_walk(vel=10.0, time=1.0)
Result: {"status": "failed", "reason": "10m/s exceeds
limit"}
Pupper: "I cannot do spider walk that fast!"
```

## How the Protocol Works

**1. Declare Function Spec**
Harness puts function args & units in System

**2. Model Emits <function_call>**
Still just next-token text prediction!

**3. Harness Runs Python**
Executes run_forward(1.0, 1.0) on hardware

**4. Return Result → Reply**
Appends JSON status; model confirms aloud

# Building a Robot Agent (1/2): Timing & Hierarchy

## 1. Async & Interruptible Actions



✕ **Blocking Call:** move(5s) freezes loop → deaf to "Stop!"

✓ **Async Task:** **Run in background → cancel / steer anytime!**

## 2. Slow Brain + Fast Body

**Slow Brain — Model + Harness**

Voice • Open-World Vision • Task Planning

**Skill Call** ↓ ↑ **State / Done**

**Fast Body — Onboard Controllers**

Pupper RL Policy • Joint Control

✕ **Never put cloud VLM in a 100 Hz torque loop!**

✓ **Expose skills:** `walk(),spider(),stand()`

**Rule of Thumb:** Async Harness steers **high-level skills** • Onboard controllers handle **real-time control**.

# Building a Robot Agent (2/2): Context & Safety

## 3. Curate Multimodal Context




✕ 30 FPS video floods context & spikes latency

✓ ~1 FPS is a good balance

✓ Attach state (`pose, battery, fault`) to give feedback!

## 4. Safety in Harness, Not Prompt




✕ Model output are probabilistic. Constraints in prompt are not always enforced

✓ Clamp speed, joints & workspace in code

✓ Timeout → e.g. be aware of Wi-Fi lag

**Takeaway: Agent = Model + Harness**

Model (Reasoning) + Async, Safe Harness (Your Code) + Robot Skills (RL)

# Summary

**1. What is a Robot Foundation Model?**

A generalist AI brain uniting internet-scale visual & language understanding with physical robot control.

**2. Why Robot Foundation Models?**

They bring world knowledge and broad generalization to messy human spaces—without task specific engineering.

**3. How Are They Built & Deployed?**

Built on a VLM backbone, wrapped in Agent Harness for high-level reasoning and tool use.

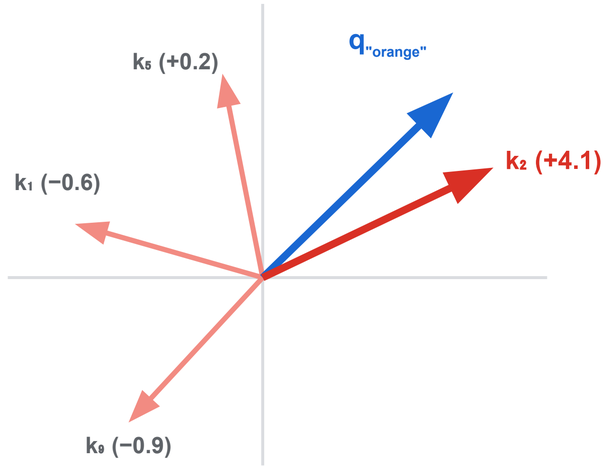
**Next Up: Your CS 123 Final Projects!**

Foundation model brain + RL walking controller + your **creative ideas**—wait to see what you build!

# Self-Attention in 3 Steps: Score → Softmax → Blend

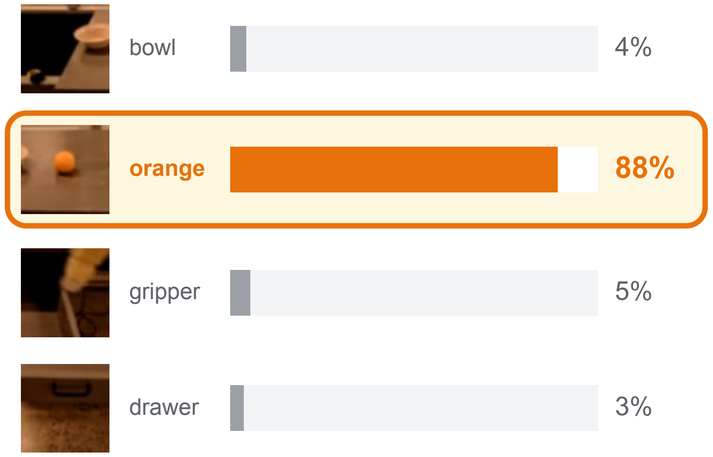
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

## Step 1 — Score ($q \cdot k_j / \sqrt{d_k}$)



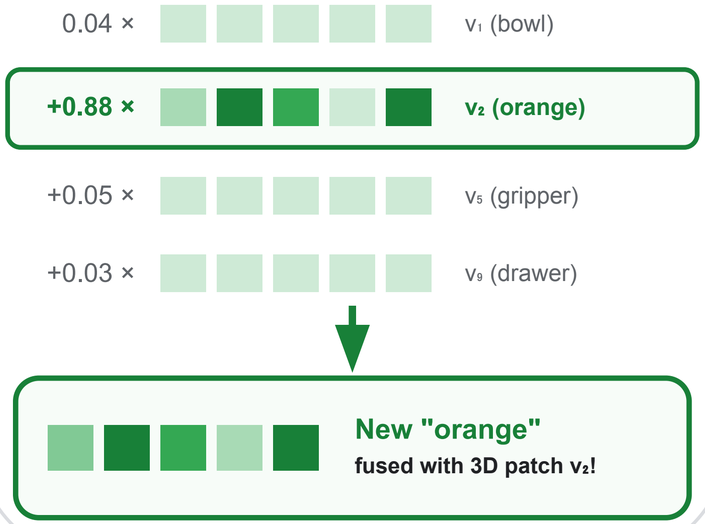

Same direction ⇒ high positive score!

## Step 2 — Softmax (Spotlight %)



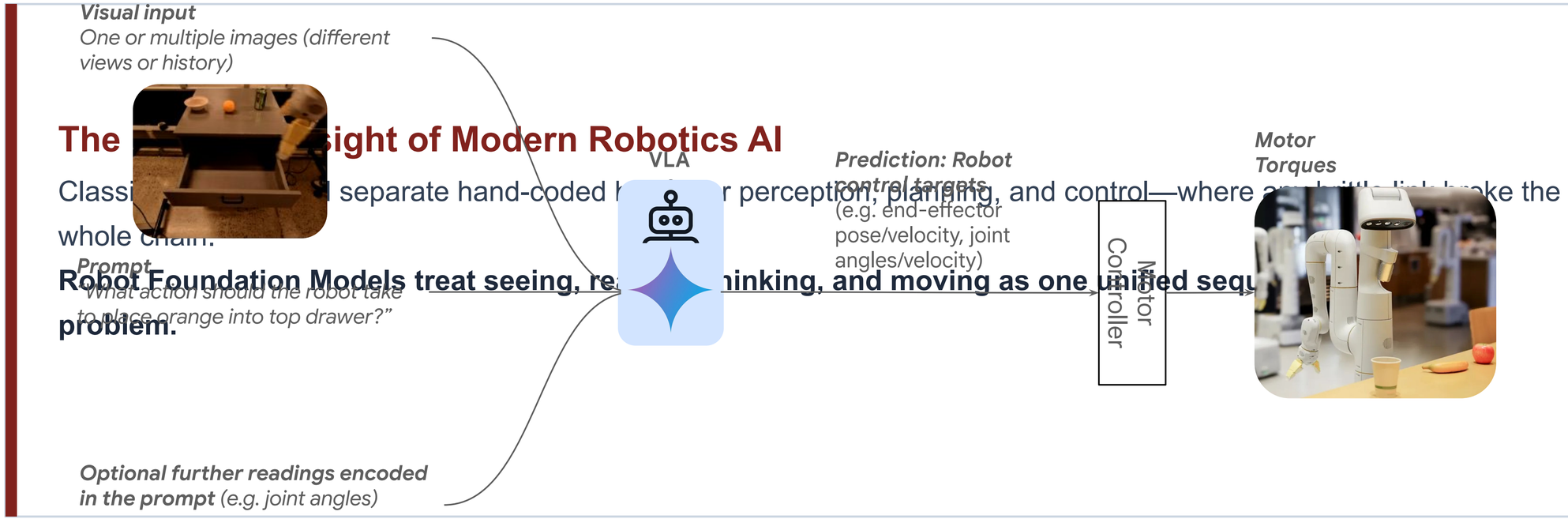

÷ $\sqrt{d_k}$ keeps weights sharp; sum = 100%

## Step 3 — Blend Values ($\sum \alpha_j v_j$)




**GPU Friendly:** Across all tokens at once, **Steps 1–3** compile into **two dense matrix multiplications!**

# Under the Hood: One Sequence Game




### 1. Senses & Goal

Live camera frames
+ English command
+ Joint sensors

### 2. Tokenization

Translate words, picture tiles & motion into one alphabet

### 3. Transformer

Cut frames into tiles; dynamic spotlight links words to 3D objects

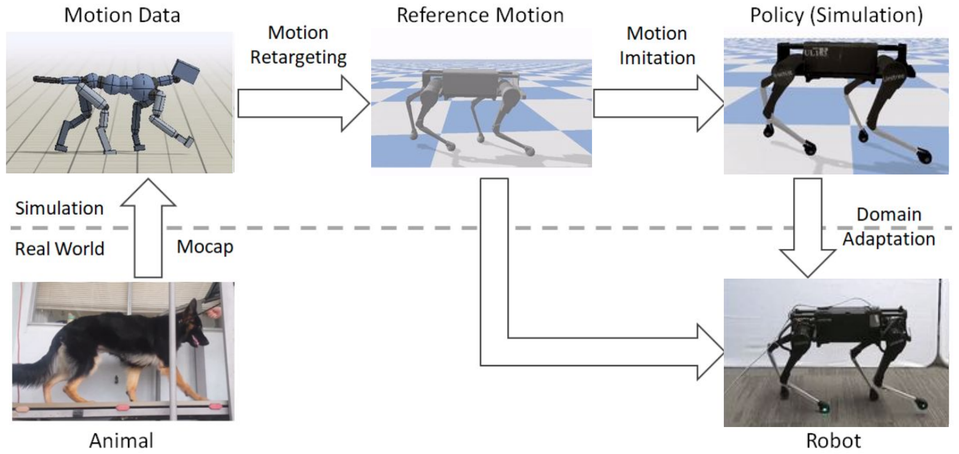
### 4. Action Head

Output smooth motions executable on the robot

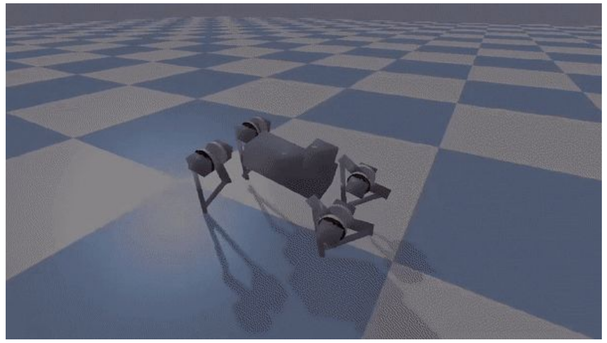
# In the last decade…



Arm Farm, 2016

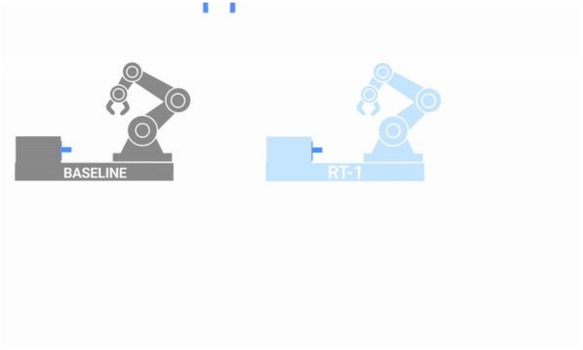



Imitating Animals, 2020





Sim-to-Real, 2018

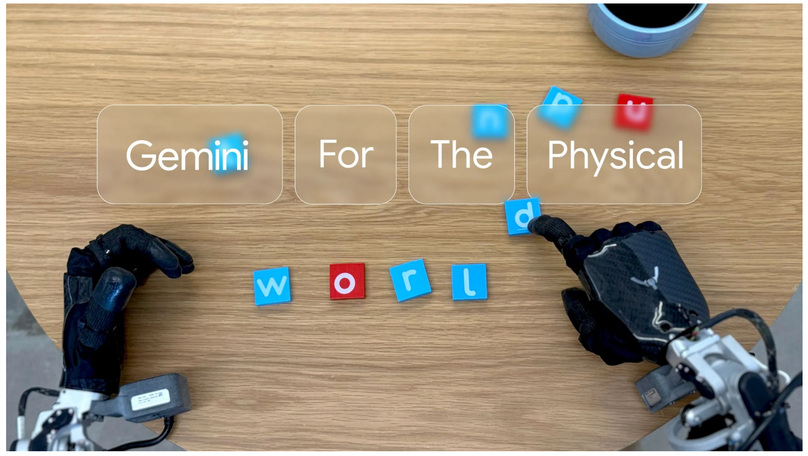



RT-1, 2022






RT-2, 2023



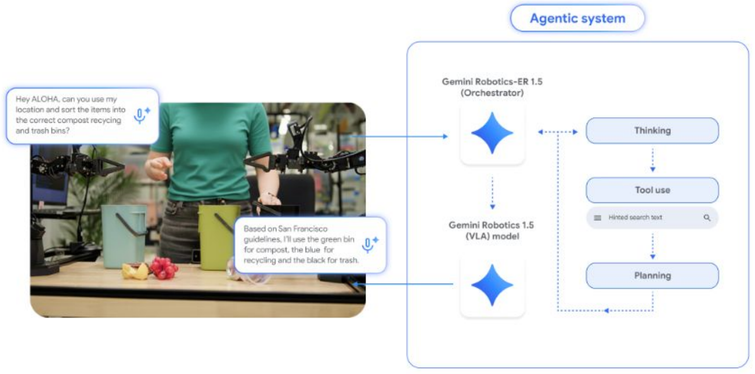

Gemini Robotics, 2025



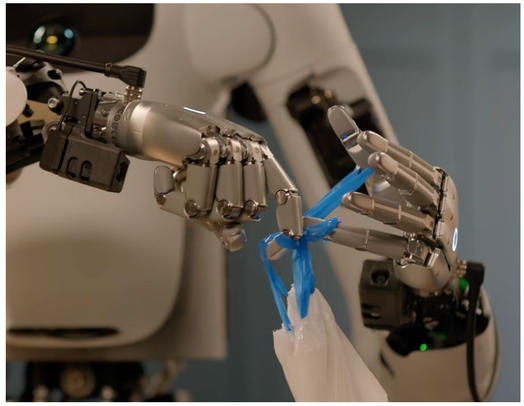

SayCan, 2022



RT-x, 2023

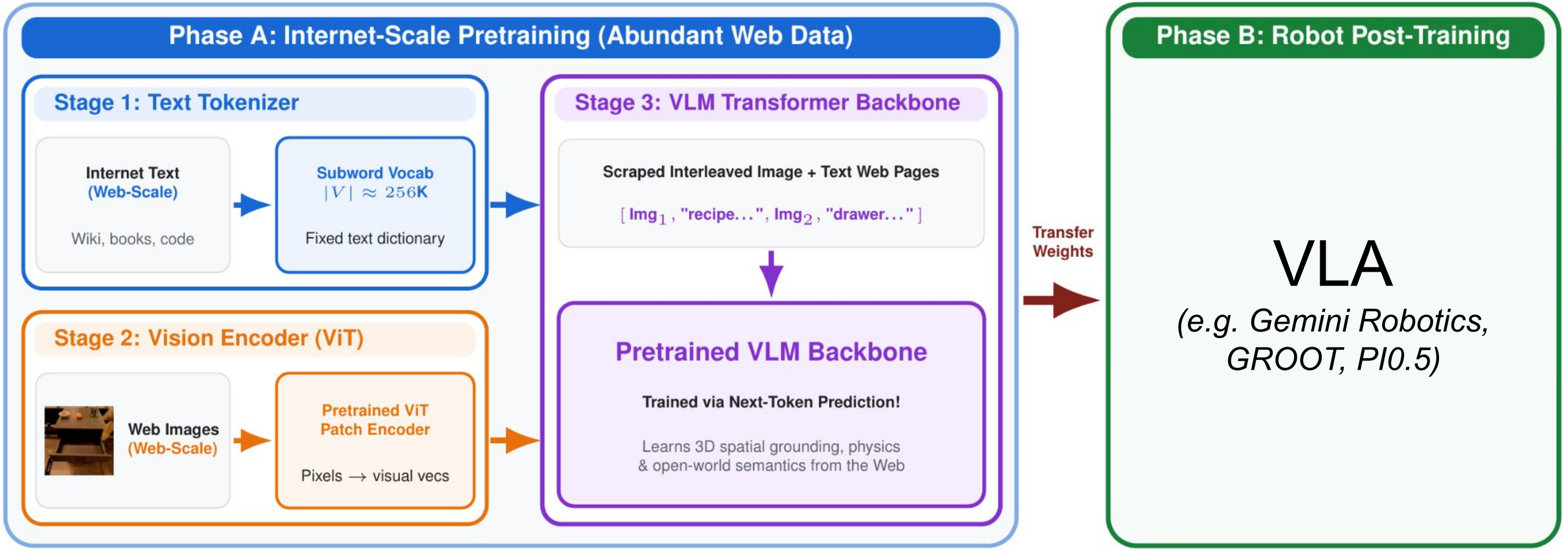



Gemini Robotics 1.5, 2025



Gemini Robotics 2, 2026

# Putting everything together: Web Pretraining → Robot Actions




**4-Stage Recipe:** Pretrain **text tokenizer**, **ViT** & **VLM** on web data first — then add **robot action tokens!**

# Vision-Language-Action Model

***Visual input***
*One or multiple images (different views or history)*



***Prompt***
*“What action should the robot take to place orange into top drawer?”*

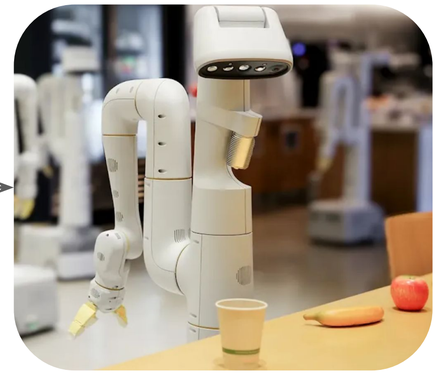
***Optional further readings encoded in the prompt*** *(e.g. joint angles)*

VLA

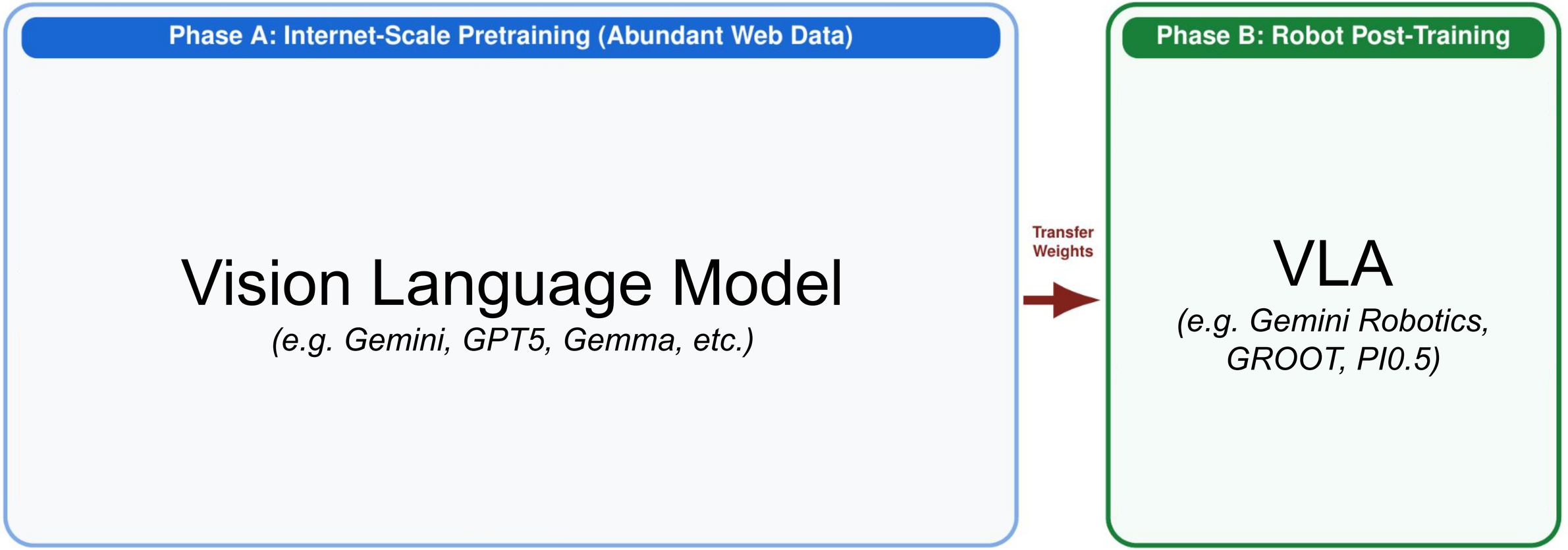
***Prediction: Robot control targets***
(e.g. end-effector pose)

Motor Controller

***Motor Torques***



# VLA: Web Pretraining → Robot Actions




**2-Stage Recipe:** Pretrain an **VLM** — then postrain with **robot action data!**

# Vision-Language-Action Model

***Visual input***
*One or multiple images (different views or history)*



***Prompt***
*"What action should the robot take to place orange into top drawer?"*

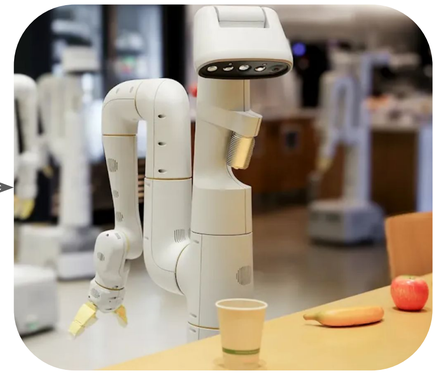
***Optional further readings encoded in the prompt*** *(e.g. joint angles)*

VLA

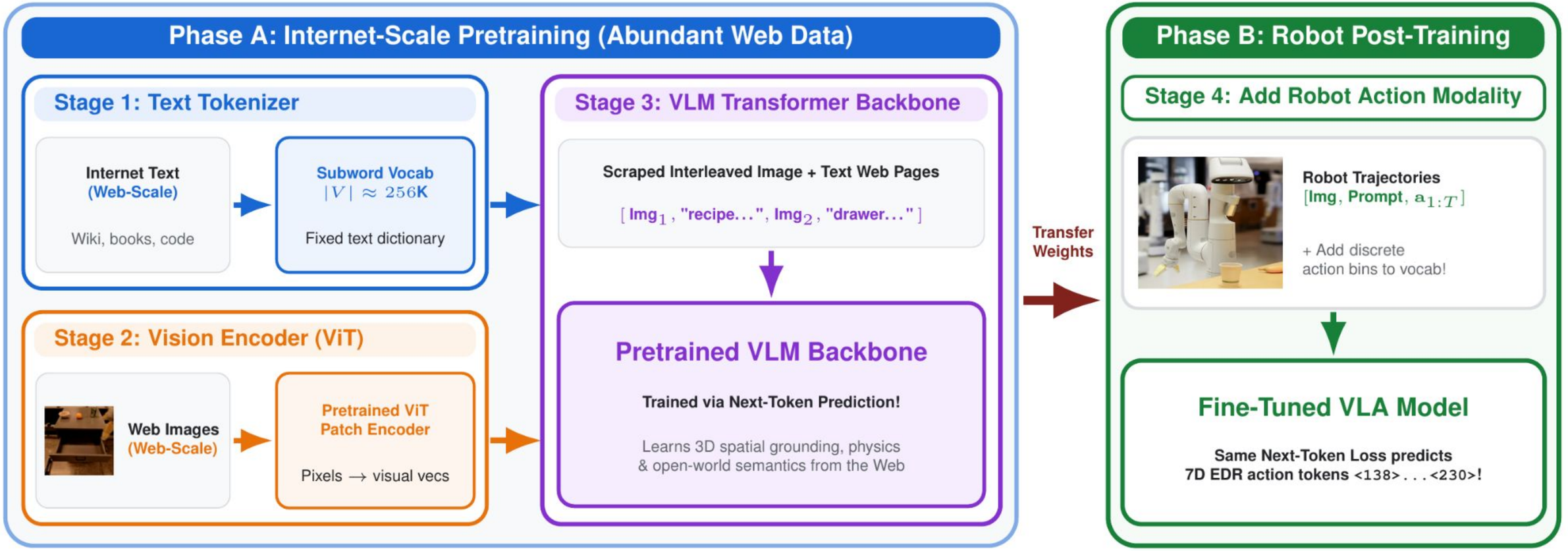
***Prediction: Robot control targets***
(e.g. end-effector pose)

Motor Controller

***Motor Torques***



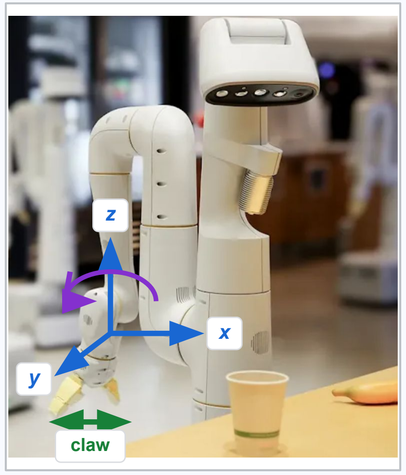
# Putting everything together: Web Pretraining → Robot Actions




**4-Stage Recipe:** Pretrain **text tokenizer**, **ViT** & **VLM** on web data first — then add **robot action tokens!**

# From Motion to Tokens: 7D Robot Action Space

## 1. Action Space (7 Continuous DoF)




**3 Positional (x, y, z)**
End-effector 3D move
`[+0.08, −0.03, −0.12]`

**3 Rotational (r, p, y)**
Wrist axis-angle orient.
`[+0.02, +0.15, −0.05]`

**1 Gripper (g)**
Open / close claw
`[0.90 = open]`

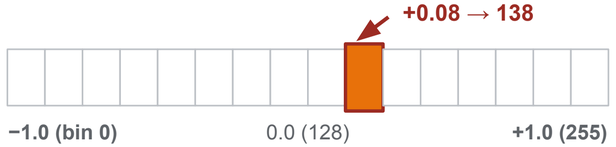
**Continuous Action Vector** $a_t \in \mathbb{R}^7$

$$[\ \Delta x, \Delta y, \Delta z\ ,\ \ \Delta r, \Delta p, \Delta y\ ,\ \ g\ ]$$

3 pos | 3 rot | 1 grip

## 2. Discretize Each Dim (256 Bins)

**Split [−1, +1] into 256 Uniform Bins (0 … 255)**




Same 256-bin grid applied to each of the 7 dims!

**7 Floats ⟶ 7 Bin IDs (0 … 255)**

| | | |
|---|---|---|
| Δx | +0.08 | bin 138 |
| Δy | −0.03 | bin 124 |
| Δz | −0.12 | bin 112 |
| Δr | +0.02 | bin 130 |
| Δp | +0.15 | bin 147 |
| $\Delta y_w$ | −0.05 | bin 121 |
| *g* | 0.90 | bin 230 |

## 3. LLM Action Tokens

**Reserve 256 IDs in LLM Vocab**

Text Tokens: "Place", "orange", …

+ 7 Action tokens: <a_0> … <a_6>

**1 Step = 7 Action Tokens!**

| | | | | | | |
|---|---|---|---|---|---|---|
| <138> Δx | <124> Δy | <112> Δz | <130> Δr | <147> Δp | <121> $\Delta y_w$ | <230> *g* |

**Predicted just like text tokens**

# Turning a VLM into a VLA: Predicting Action Tokens

### 1. Multimodal Context Tokens



Image Patches
$[v_1 \dots v_8]$

**Prompt Tokens**
**"Place orange into drawer"**

Interleaved 1D sequence (from Slide 19)
$[v_1, \dots, v_8$, "Place", "orange", "into", "drawer"]

### 2. Same Autoregressive Transformer (VLA)

**Next-Token Prediction**
Predicts 1 action bin per step
(7× steps per robot command)

**Zero change to LLM architecture!**

### 3. Output: 7 Discrete Action Tokens

| 138 | 124 | 112 | 130 | 147 | 121 | 230 |
|---|---|---|---|---|---|---|
| Δx | Δy | Δz | Δr | Δp | $\Delta y_w$ | *g* |

3 Position Bins — 3 Rotation Bins — Gripper

### 4. De-Tokenize (Bin → Float)

**Lookup 7D bin centers:**
**Pos: (+0.08, −0.03, −0.12)**
**Rot: (+0.02, +0.15, −0.05)  Grip: 0.90**

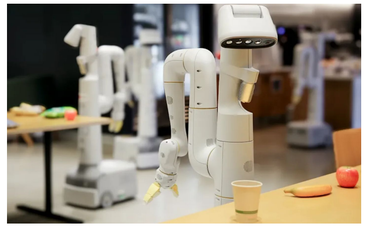
Continuous 7D End-Effector Target

### 5. Motor Controller

**Inverse Kinematics + High-Freq PD**

Fast low-level servo control loop

Torques



### 6. EDR Robot Moves Arm

Reaches for orange; captures next camera frame

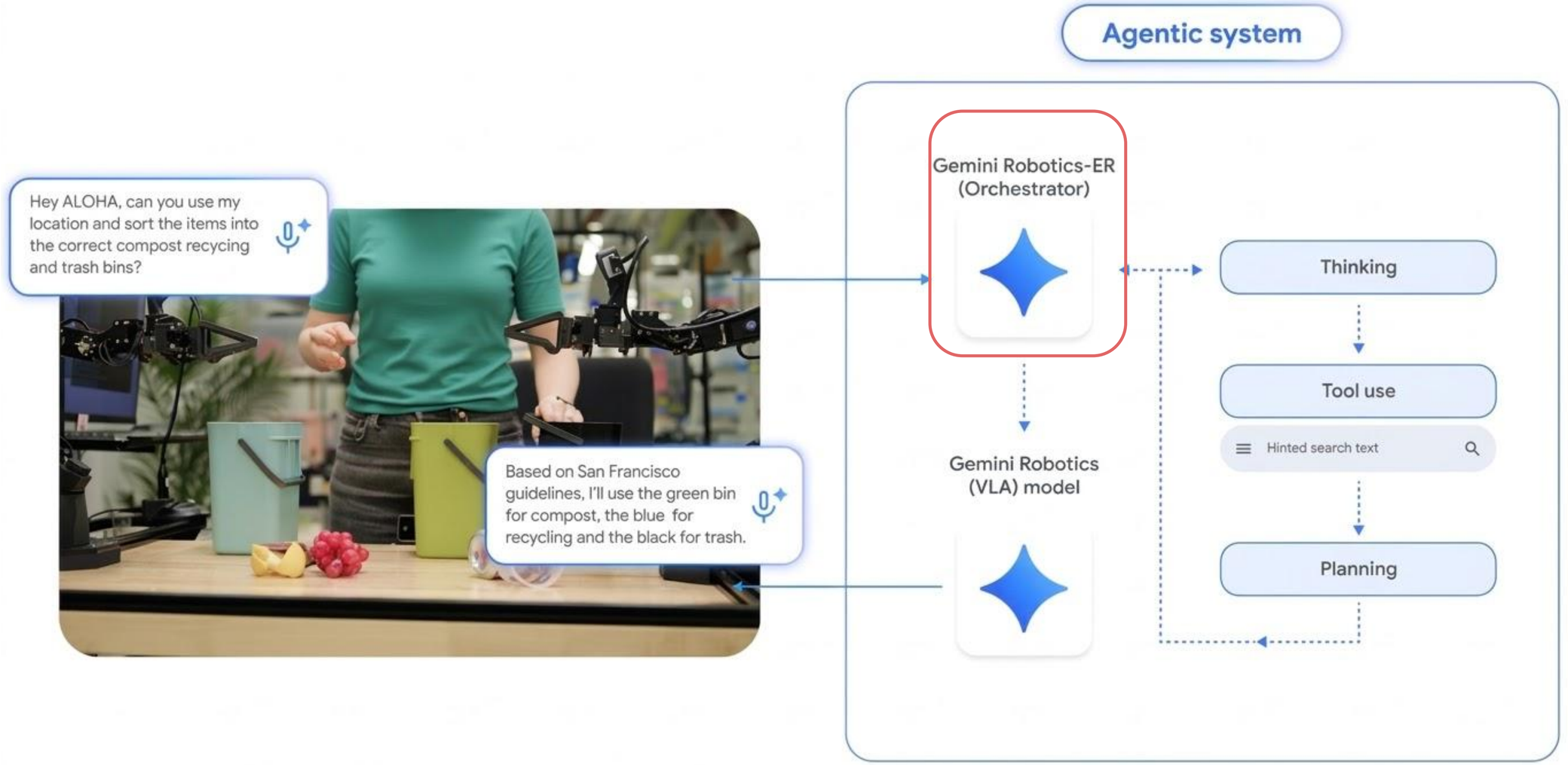
Closed-Loop Next Frame

**VLM → VLA:** Predicting **discrete action tokens** is standard **next-token prediction** — **zero architecture change!**

# Gemini Robotics



# How does it work?



# The Granularity Trade-off: Characters vs. Words vs. Subwords

How should we tokenize **"unbelievably"**?

## 1. CHARACTER / BYTE LEVEL

```
['u','n','b','e','l','i',
 'e','v','a','b','l','y']
```

**Vocab |V|:** Tiny (~256 UTF-8 bytes)
**Seq Length:** **12 tokens (4–5× longer!)**

**+ Pros:** Zero unknown words
**– Cons:** Attention is $O(N^2)$ ⟶ ~20× more compute! Single letters ('u') carry almost no meaning.

## 2. WHOLE-WORD LEVEL

```
["unbelievably"]
```

**Vocab |V|:** Huge (500K – 2M+ words)
**Seq Length:** 1 token (very compact)

**+ Pros:** Short sequences; strong word-level semantics.
**– Cons:** Out-of-vocab [UNK] on new words, code, or typos!

## 3. SUBWORDS ★ MODERN LLMs

`un` `believ` `ably`

**Vocab |V|:** **32K – 256K (Sweet Spot!)**
**Seq Length:** ~1.3 tokens / English word

**★ Best of Both Worlds:**
• Common words stay intact ("␣the")
• Rare words split into meaningful roots + affixes (un + believ + ably)

**Every design balances Vocabulary Size |V| against Sequence Length N.**

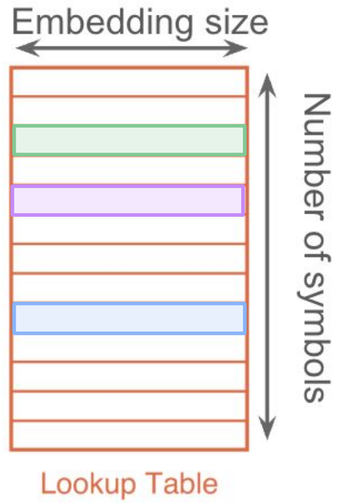
**Modern Vocab Sizes |V|:** GPT-2 **50K** • GPT-4 **100K** • Llama 3 **128K** • GPT-4o **200K** • Gemini / Gemma **256K**

# From Token IDs to Embeddings: Entering & Exiting the Transformer

**Why Not Feed Integer IDs Directly?** Token IDs are arbitrary dictionary indices! If `"cat" = 3797` and `"dog" = 5679`, 5679 > 3797 does NOT mean "dog > cat". Instead, each ID indexes a row in a **learnable Embedding Matrix E**.

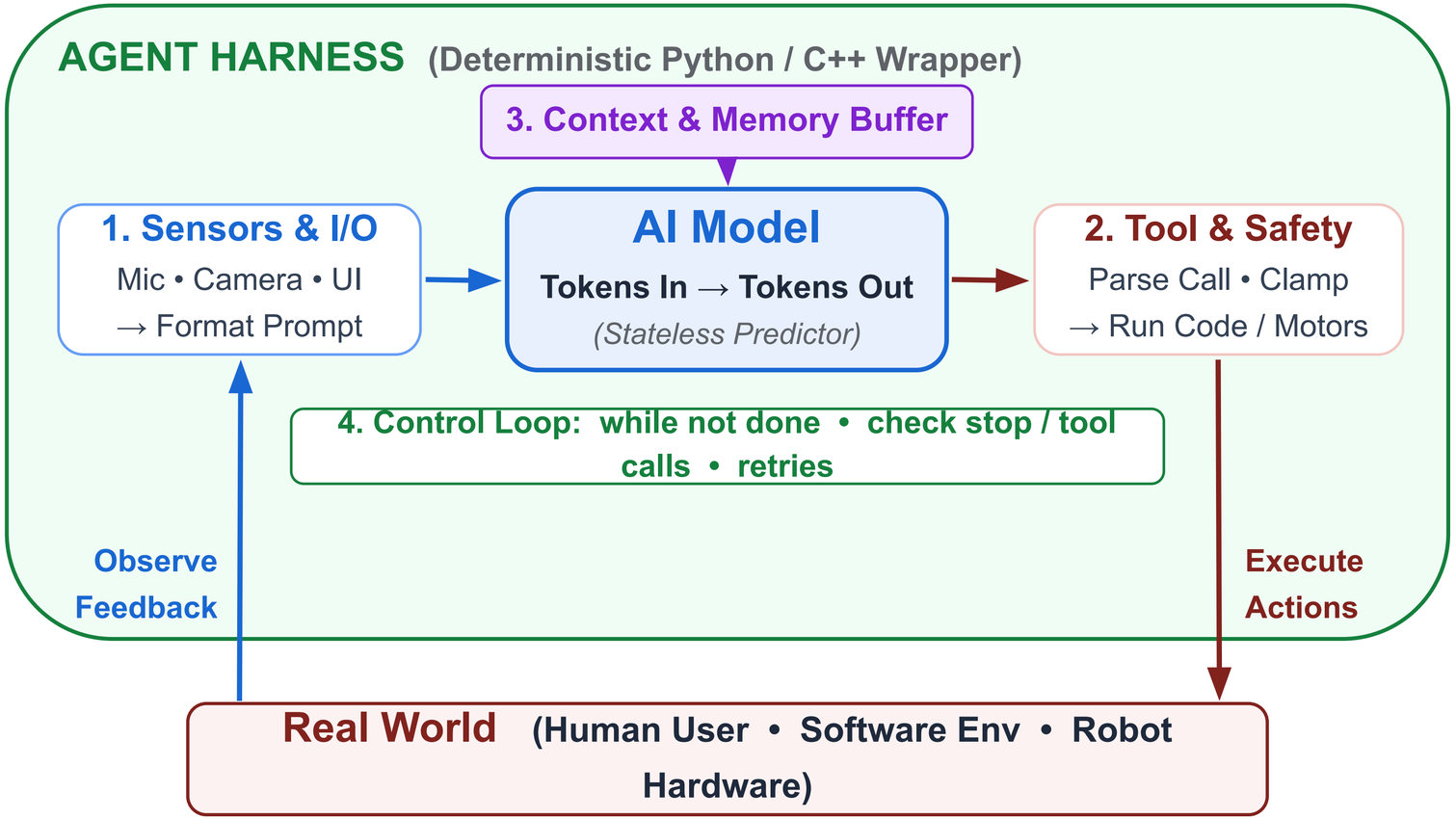
**EMBEDDING LOOKUP TABLE** `E ∈ ℝ^(|V| × d_model)` `(e.g., 200K × 4096)`

| | |
|---|---|
| `"Place" → 32371` | `Row 32371: [ 0.04, -0.71, 0.33, …, 0.19 ]` |
| `"␣orange" → 10214` | `Row 10214: [ 0.88, 0.12, -0.45, …, -0.62 ]` |
| `"␣drawer" → 26357` | `Row 26357: [-0.25, 0.64, 0.51, …, 0.08 ]` |




**Where Do Embeddings Come From?** Matrix **E** starts random and is learned via gradient descent during pretraining! Tokens used in similar contexts (`"␣orange"`, `"␣tangerine"`) naturally converge to nearby directions in $\mathbb{R}^d$.

# What Is an Agent Harness?




## Definition

Everything **besides the AI model** that turns token predictions into real-world loops, memory, and actions.

*Anthropic (2024/2025) • Google "Agents" (2024)*

## What You Actually Build

- **AI Model:** Off-the-shelf API / weights
- **Harness:** **The robotics & systems code YOU write!**

**Bottom Line:** The AI Model only sees and emits tokens — **the Harness bridges tokens ↔ the real world!**

# Example 2: Function Calling Under the Hood

## Complete Context Transcript

*# 1. Harness declares tool + units in System prompt*

**System:** You are Pupper, a friendly robot dog.

*# 2. Speech turn*

**User:** 🎤 Bark!

**Dog:** 📣 Woof! Woof!

## How the Protocol Works

**1. Declare Tool Spec**
Harness puts function args & units in System

**2. Model Emits <action>**
Still just next-token text prediction!

**3. Harness Runs Python**
Executes run_forward(1.0, 1.0) on hardware

**4. Return Result → Reply**
Appends JSON status; model confirms aloud

**Zero Magic:** Model outputs **structured text** • Harness executes **real Python** • **Modern APIs build this in natively!**

# Example 3: Robot Agent Code (1/2 — Setup)

```python
# 1. Robot movement control function (Hardware / RL API)
def move_robot(direction: str, distance: float = 1.0) -> dict:
  """Controls robot base movement.
  Args:
    direction: 'forward', 'backward', 'left', or 'right'.
    distance: Distance in meters to move.
  """
  print(f"[ROBOT HARDWARE]: Moving {direction} {distance}m")
  return {"status": "success", "move": direction, "m": distance}

# 2. Register tool & configure Gemini Live session
client = genai.Client()
model = "gemini-3.1-flash-live-preview"
tool_decl = types.FunctionDeclaration.from_callable(
    client=client._api_client, callable=move_robot
)
config = types.LiveConnectConfig(
    response_modalities=["AUDIO"],
    output_audio_transcription=types.AudioTranscriptionConfig(),
    tools=[types.Tool(function_declarations=[tool_decl])],
    system_instruction="You are a robot assistant. When asked "
        "to move, call move_robot and briefly report back.",
)
```

### ◀ Docstring + Types = Tool Spec

- Valid options + units (meters!)

### ◀ Hardware Hook

- Swap print() → ROS2 / Pupper RL

### ◀ Gemini Live Config

- Native AUDIO + transcription + tools

**Tip:** `from_callable()` turns your Python type hints & docstring directly into the model's tool schema.

# Example 3: Robot Agent Code (2/2 — Loop)

```python
async with client.aio.live.connect(model=model, config=config) as session:
  while True:  # <--- THE HARNESS CONTROL LOOP
    user_input = input("User > ").strip()  # or stream mic/camera
    await session.send_client_content(
        turns=[types.Content(role="user", parts=[types.Part(text=user_input)])],
        turn_complete=True,
    )
    async for chunk in session.receive():
      # A. Stream spoken audio & text transcript
      if chunk.server_content and chunk.server_content.output_transcription:
        print(chunk.server_content.output_transcription.text, end="")

      # B. Intercept tool_call → execute on robot → return result!
      elif chunk.tool_call:
        for call in chunk.tool_call.function_calls:
          res = move_robot(**call.args)
          await session.send_tool_response(function_responses=[{
              "id": call.id, "name": call.name, "response": {"result": res}
          }])
      if chunk.server_content and chunk.server_content.turn_complete:
        break
```

## Live Turn Flow

**1. Input:** "Move forward 2m"

↓

**2. tool_call:** move_robot('forward', 2.0)

↓

**3. Robot Act:** Motors move → {"status": "success"}

↓

**4. Return:** send_tool_response(res)

↓

**5. Speak:** 🔊 "Done! Moved 2 meters."

**Complete Working Agent!** Now: what **4 things** must you keep in mind on a **real physical robot**?